Banco de Dados na Prática: Dashboard de Vendas com SQLite e Streamlit

Antes de começar

O que vamos construir

Vamos transformar uma planilha de vendas em um **projeto de banco de dados completo**: modelar as tabelas, carregar os dados com um script, escrever consultas SQL e mostrar tudo em um dashboard web feito com Streamlit. No final, o projeto estará publicado na internet e pronto para entrar no seu portfólio.

A planilha `system_extraction.xlsx` (aba `salesreport`) tem 4.384 pedidos de 2021, com vendedor, cliente, produto, região, quantidade, preço e margem de lucro.

Objetivos de aprendizagem

Ao final desta aula você será capaz de:

- explicar por que dados repetidos em uma planilha viram problema e como o modelo relacional resolve;
- criar um esquema de banco de dados com chaves primárias, chaves estrangeiras, restrições, índices e uma view;
- escrever um script ETL que lê Excel e popula um banco SQLite;
- escrever consultas com `JOIN`, `GROUP BY`, subquery, CTE e window functions;
- construir um dashboard interativo em Streamlit que consulta o banco;
- versionar com Git e publicar o app no Streamlit Community Cloud.

Pré-requisitos

- Python 3.10 ou superior instalado (o material foi testado com 3.12 e 3.13);
- Git instalado e uma conta no GitHub;
- um editor de código (VS Code é uma boa escolha);
- o arquivo `system_extraction.xlsx`, entregue pelo professor.

Duração sugerida: 3 a 4 horas, podendo ser dividida em duas aulas (Partes 1 a 6 na primeira, Partes 7 a 9 na segunda).

Parte 1 — Conceitos que você precisa

Por que não deixar tudo na planilha? Na planilha, o texto `Produto 2` aparece escrito centenas de vezes. Se alguém digitar `Produto 2` com dois espaços, ele vira outro produto. Nada impede um preço negativo, um pedido duplicado ou um vendedor inexistente. Um banco de dados resolve isso com **regras que o próprio banco faz cumprir**.

Banco relacional. Os dados ficam em *tabelas* ligadas entre si. Cada linha de uma tabela é identificada por uma **chave primária (PK)**. Para apontar para uma linha de outra tabela, usamos uma **chave estrangeira (FK)**.

Normalização. É a técnica de separar o que se repete. Em vez de gravar o nome do cliente em cada pedido, criamos uma tabela `cliente` e o pedido guarda só o `cliente_id`. As tabelas pequenas e descritivas (cliente, produto, vendedor, região) são **dimensões**. A tabela grande com os eventos (pedido) é a tabela de **fatos**.

Valor derivado. Na planilha, `Valor Pedido` é sempre `Quantidade × Preço`. Guardar um valor que dá para calcular abre espaço para inconsistência (alguém altera a quantidade e esquece o valor). Por isso ele **não** vira coluna: será calculado em uma **view**.

View. É uma consulta salva com nome. Ela se comporta como uma tabela, mas não guarda dados próprios.

ETL. *Extract, Transform, Load*: extrair dados da origem (Excel), transformá-los (limpar, converter datas) e carregá-los no destino (banco).

SQLite. Um banco relacional completo que vive em um único arquivo (`.db`). Não precisa instalar servidor, e o Python já traz suporte a ele.

Streamlit. Biblioteca que transforma um script Python em uma página web interativa.

Pense: em uma tabela `pedido` que guardasse o texto `Paulo` em vez de `vendedor_id`, o que aconteceria se o Paulo mudasse de nome?

Parte 2 — Preparando o ambiente

Passo 1: criar a pasta e o ambiente virtual

Explicação: o *ambiente virtual* isola as bibliotecas deste projeto das outras coisas instaladas no seu computador.

Faça: abra o terminal e rode.

```
mkdir projeto_vendas
cd projeto_vendas
python -m venv .venv
```

Ative o ambiente:

```
# Windows (PowerShell)
.venv\Scripts\Activate.ps1

# Linux ou macOS
source .venv/bin/activate
```

Confira: o início da linha do terminal passa a mostrar `(.venv)`.

> **Atenção:** a pasta `.venv` e o arquivo `pyvenv.cfg` **nunca** vão para o Git. Eles contêm caminhos do seu computador e o nome do seu usuário.

Passo 2: instalar as bibliotecas

Crie o arquivo `requirements.txt` com este conteúdo:

```
streamlit>=1.35
pandas
altair
openpyxl
```

E instale:

```
pip install -r requirements.txt
```

O `openpyxl` é o que permite ao pandas ler arquivos `.xlsx`.

Passo 3: organizar as pastas

```
projeto_vendas/
├── app.py
├── requirements.txt
├── README.md
├── .gitignore
├── data/
│   └── system_extraction.xlsx
├── sql/
│   ├── schema.sql
│   └── queries.sql
└── src/
    └── etl.py
```

Crie as pastas `data`, `sql` e `src`, e mova o Excel para `data/`. Crie também o `.gitignore`:

```
.venv/
venv/
pyvenv.cfg
__pycache__/
db/
.streamlit/secrets.toml
```

A pasta `db/` fica de fora do Git porque o banco é **gerado** pelo script. Quem clonar o projeto gera o seu próprio.

Parte 3 — Conhecendo os dados

Passo 4: explorar a planilha antes de modelar

Explicação: modelar sem olhar os dados é chute. Vamos descobrir quais colunas se repetem (candidatas a dimensão) e se há sujeira.

Faça: crie um arquivo temporário `explorar.py` na raiz e rode `python explorar.py`.

```
import pandas as pd

df = pd.read_excel("data/system_extraction.xlsx", sheet_name="salesreport")
print(df.shape)
print(df.dtypes)
print(df.isna().sum())
for col in ["Vendedor", "Produto vendido", "Região", "Cliente"]:
    print(col, df[col].nunique(), sorted(df[col].unique()))
print("Nº pedido único?", df["Nº pedido"].is_unique)
```

Confira:

- formato `(4384, 10)`;
- nenhuma célula vazia;
- 4 vendedores, 5 produtos, 8 regiões e 10 clientes;
- `Nº pedido único? True`.

Pense: quais colunas têm poucos valores diferentes repetidos milhares de vezes? Essas viram tabelas de dimensão. E qual coluna identifica cada linha de forma única? Essa é a chave primária do pedido.

Parte 4 — Modelando o banco

Passo 5: escrever o `sql/schema.sql`

Explicação: o esquema é a planta da casa. Aqui definimos as tabelas e as regras. Leia os comentários com atenção, porque cada decisão tem um motivo.

Faça: crie `sql/schema.sql`.

```
PRAGMA foreign_keys = ON;

CREATE TABLE vendedor (
    id   INTEGER PRIMARY KEY,
    nome TEXT NOT NULL UNIQUE
);

CREATE TABLE cliente (
    id   INTEGER PRIMARY KEY,
    nome TEXT NOT NULL UNIQUE
);

CREATE TABLE produto (
    id   INTEGER PRIMARY KEY,
    nome TEXT NOT NULL UNIQUE
);

CREATE TABLE regiao (
    id   INTEGER PRIMARY KEY,
    nome TEXT NOT NULL UNIQUE
);

CREATE TABLE pedido (
    numero_pedido INTEGER PRIMARY KEY,
    data          TEXT    NOT NULL,              -- ISO: AAAA-MM-DD
    vendedor_id   INTEGER NOT NULL REFERENCES vendedor(id),
    cliente_id    INTEGER NOT NULL REFERENCES cliente(id),
    produto_id    INTEGER NOT NULL REFERENCES produto(id),
    regiao_id     INTEGER NOT NULL REFERENCES regiao(id),
    quantidade    INTEGER NOT NULL CHECK (quantidade > 0),
    preco         REAL    NOT NULL CHECK (preco > 0),
    margem_lucro  REAL    NOT NULL
);

CREATE INDEX idx_pedido_data     ON pedido(data);
CREATE INDEX idx_pedido_vendedor ON pedido(vendedor_id);
CREATE INDEX idx_pedido_cliente  ON pedido(cliente_id);
```

```
CREATE INDEX idx_pedido_produto  ON pedido(produto_id);

-- Valor do pedido é derivado (quantidade x preço): fica na view, não na
tabela.
CREATE VIEW vw_vendas AS
SELECT p.numero_pedido,
       p.data,
       v.nome AS vendedor,
       c.nome AS cliente,
       pr.nome AS produto,
       r.nome AS regiao,
       p.quantidade,
       p.preco,
       ROUND(p.quantidade * p.preco, 2) AS valor_pedido,
       p.margem_lucro
FROM pedido p
JOIN vendedor v ON v.id = p.vendedor_id
JOIN cliente  c ON c.id = p.cliente_id
JOIN produto pr ON pr.id = p.produto_id
JOIN regiao   r ON r.id = p.regiao_id;
```

Entendendo o código:

- `PRIMARY KEY` identifica cada linha de forma única.
- `NOT NULL UNIQUE` em `nome` impede nomes vazios e repetidos nas dimensões.
- `REFERENCES` cria a chave estrangeira. Com `PRAGMA foreign_keys = ON`, o banco recusa um pedido apontando para um cliente que não existe.
- `CHECK` valida o dado: não aceita quantidade ou preço menores ou iguais a zero.
- Os **índices** aceleram buscas e filtros nas colunas mais usadas (data e chaves estrangeiras).
- A data é guardada como texto `AAAA-MM-DD`. Nesse formato, a ordem alfabética coincide com a ordem cronológica, e o SQLite entende funções como `strftime`.

Diagrama do modelo (cada seta é uma relação 1 para N):

```
vendedor ──┐
cliente  ──┼──< pedido
produto  ──┤
regiao   ──┘
```

Pense: por que o `valor_pedido` está na view e não na tabela `pedido`?

Parte 5 — Carregando os dados (ETL)

Passo 6: escrever o `src/etl.py`

Explicação: o script lê o Excel, limpa, cria o banco a partir do `schema.sql` e insere as linhas. Ele apaga e recria o banco a cada execução, então rodar duas vezes não duplica nada.

Faça: crie `src/etl.py`.

```
"""Lê data/system_extraction.xlsx e cria o banco SQLite db/vendas.db."""
import sqlite3
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parent.parent
XLSX = ROOT / "data" / "system_extraction.xlsx"
SCHEMA = ROOT / "sql" / "schema.sql"
DB = ROOT / "db" / "vendas.db"

COLUNAS = {
    "Data": "data", "Nº pedido": "numero_pedido", "Vendedor": "vendedor",
    "Produto vendido": "produto", "Quantidade": "quantidade", "Preço":
"preco",
    "Região": "regiao", "Cliente": "cliente", "Margem Lucro": "margem_lucro",
}


def carregar_planilha() -> pd.DataFrame:
    df = pd.read_excel(XLSX, engine="openpyxl", sheet_name="salesreport")
    df = df.rename(columns=COLUNAS)[list(COLUNAS.values())]
    df["data"] = pd.to_datetime(df["data"]).dt.strftime("%Y-%m-%d")
    for col in ("vendedor", "produto", "regiao", "cliente"):
        df[col] = df[col].astype(str).str.strip()
    if df["numero_pedido"].duplicated().any():
        raise ValueError("Há números de pedido duplicados na planilha.")
    return df


def main() -> None:
    df = carregar_planilha()
    DB.parent.mkdir(exist_ok=True)
    DB.unlink(missing_ok=True)  # recria do zero, evita duplicar linhas

    con = sqlite3.connect(DB)
```

```
    con.executescript(SCHEMA.read_text(encoding="utf-8"))

    ids = {}
    for dim in ("vendedor", "cliente", "produto", "regiao"):
        nomes = sorted(df[dim].unique())
        con.executemany(f"INSERT INTO {dim}(nome) VALUES (?)", [(n,) for n in
nomes])
        ids[dim] = dict(con.execute(f"SELECT nome, id FROM {dim}"))

    linhas = [
        (int(r.numero_pedido), r.data,
         ids["vendedor"][r.vendedor], ids["cliente"][r.cliente],
         ids["produto"][r.produto], ids["regiao"][r.regiao],
         int(r.quantidade), float(r.preco), float(r.margem_lucro))
        for r in df.itertuples(index=False)
    ]
    con.executemany(
        """INSERT INTO pedido(numero_pedido, data, vendedor_id, cliente_id,
                              produto_id, regiao_id, quantidade, preco,
margem_lucro)
           VALUES (?,?,?,?,?,?,?,?,?)""", linhas)
    con.commit()
    total = con.execute("SELECT COUNT(*) FROM pedido").fetchone()[0]
    con.close()
    print(f"Banco criado em {DB} com {total} pedidos.")


if __name__ == "__main__":
    main()
```

Entendendo o código, em ordem:

1. **Extract:** `carregar_planilha` lê a aba `salesreport`.
2. **Transform:** renomeia as colunas para nomes sem acento ou espaço, converte a data para `AAAA-MM-DD`, remove espaços sobrando nos textos e **para tudo** se houver pedido duplicado.
3. **Load, dimensões primeiro:** insere os valores únicos de vendedor, cliente, produto e região e guarda o `id` gerado em um dicionário.
4. **Load, fatos depois:** cada pedido troca os nomes pelos ids correspondentes. Por isso as dimensões precisam existir antes.
5. Os `?` são **parâmetros**: o valor nunca é colado no texto do SQL. É assim que se evita SQL injection.

Faça: rode na raiz do projeto.

```
python src/etl.py
```

Confira: deve aparecer `Banco criado em ... com 4384 pedidos.` Agora valide os totais direto no banco:

```
python -c "import sqlite3; c=sqlite3.connect('db/vendas.db');
print(c.execute('SELECT COUNT(*), ROUND(SUM(valor_pedido),2),
ROUND(SUM(margem_lucro),2) FROM vw_vendas').fetchone())"
```

Resultado esperado: `(4384, 573424.5, 88349.0)`. Se bateu, seu banco está correto.

Parte 6 — Consultas SQL

Passo 7: escrever o `sql/queries.sql`

Explicação: estas cinco consultas mostram técnicas diferentes. Elas também serão exibidas na aba **SQL ao vivo** do app. O comentário `-- name:` antes de cada uma dá o título que aparece no menu.

Faça: crie `sql/queries.sql`.

```
-- name: Faturamento e margem por vendedor
SELECT vendedor,
       COUNT(*)                                          AS pedidos,
       ROUND(SUM(valor_pedido), 2)                       AS faturamento,
       ROUND(SUM(margem_lucro), 2)                       AS lucro,
       ROUND(100.0 * SUM(margem_lucro) / SUM(valor_pedido), 1) AS margem_pct
FROM vw_vendas
GROUP BY vendedor
ORDER BY faturamento DESC;

-- name: Ranking de produtos por região (window function)
SELECT regiao, produto, faturamento,
       RANK() OVER (PARTITION BY regiao ORDER BY faturamento DESC) AS posicao
FROM (
    SELECT regiao, produto, ROUND(SUM(valor_pedido), 2) AS faturamento
    FROM vw_vendas
    GROUP BY regiao, produto
)
ORDER BY regiao, posicao;

-- name: Evolução mensal com acumulado (CTE + window function)
WITH mensal AS (
    SELECT strftime('%Y-%m', data) AS mes,
           ROUND(SUM(valor_pedido), 2) AS faturamento
```

```
    FROM vw_vendas
    GROUP BY mes
)
SELECT mes, faturamento,
       ROUND(SUM(faturamento) OVER (ORDER BY mes), 2) AS acumulado
FROM mensal
ORDER BY mes;

-- name: Clientes acima do ticket médio (subquery)
SELECT cliente,
       ROUND(AVG(valor_pedido), 2) AS ticket_medio
FROM vw_vendas
GROUP BY cliente
HAVING AVG(valor_pedido) > (SELECT AVG(valor_pedido) FROM vw_vendas)
ORDER BY ticket_medio DESC;

-- name: Melhor vendedor de cada mês (CTE + ROW_NUMBER)
WITH vendas_mes AS (
    SELECT strftime('%Y-%m', data) AS mes, vendedor,
           ROUND(SUM(valor_pedido), 2) AS faturamento
    FROM vw_vendas
    GROUP BY mes, vendedor
)
SELECT mes, vendedor, faturamento
FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY mes ORDER BY faturamento DESC)
AS rn
    FROM vendas_mes
)
WHERE rn = 1
ORDER BY mes;
```

Entendendo cada consulta:

1. **Agregação simples:** `GROUP BY vendedor` junta os pedidos de cada pessoa. `SUM`, `COUNT` e `ROUND` calculam os totais. O `100.0` força divisão decimal.
2. **Window function `RANK`:** diferente do `GROUP BY`, ela não elimina linhas. `PARTITION BY regiao` reinicia o ranking em cada região.
3. **CTE (`WITH`):** dá nome a um resultado intermediário, deixando a consulta legível. `SUM(...) OVER (ORDER BY mes)` calcula o acumulado mês a mês.
4. **Subquery:** o `HAVING` compara a média de cada cliente com a média geral, calculada na consulta interna.
5. **`ROW_NUMBER`:** numera os vendedores de cada mês do maior para o menor faturamento. O `WHERE rn = 1` fica só com o primeiro.

Confira (rode cada uma no DB Browser for SQLite ou na aba SQL do app, depois do Passo 9):

Consulta	Resultado esperado
1	4 linhas. A primeira é Ana, com 1.163 pedidos e faturamento de 158.306,55
2	40 linhas (8 regiões × 5 produtos)
3	12 linhas, uma por mês de 2021
4	6 clientes
5	12 linhas

Pense: qual é a diferença entre `GROUP BY` e `PARTITION BY`? Em qual dos dois a quantidade de linhas do resultado diminui?

Parte 7 — Construindo o app com Streamlit

Passo 8: entender por que reescrever o código original

O projeto original lia o Excel direto, sem banco. **Antes de seguir, tente achar três problemas** no código antigo (`analisys.py`). Depois compare com esta lista:

- o slider de data tinha um valor só e começava no mínimo, então o painel abria mostrando apenas o primeiro dia;
- os cartões de total ignoravam os filtros e podiam dividir por zero;
- o gráfico de vendas mensais não agregava por mês;
- a planilha era relida inteira a cada clique, e o limite `nrows=4400` estava fixo no código.

Na versão nova, os dados vêm do banco, os filtros valem para tudo e a conexão é só de leitura.

Passo 9: escrever o `app.py` em quatro blocos

Explicação: o arquivo é longo, então vamos montá-lo por partes. **Cole os blocos em sequência, no mesmo arquivo `app.py`**, sem apagar nada entre eles.

Bloco 1: configuração e acesso ao banco

```
import re
import sqlite3
import subprocess
import sys
from pathlib import Path

import altair as alt
import pandas as pd
import streamlit as st

ROOT = Path(__file__).parent
DB = ROOT / "db" / "vendas.db"
SQL_DIR = ROOT / "sql"

st.set_page_config(
    page_title="Dashboard de Vendas · Banco de Dados",
    page_icon="$",
    layout="wide",
    initial_sidebar_state="expanded",
)


@st.cache_resource
def conectar() -> sqlite3.Connection:
    if not DB.exists():  # primeiro acesso (ex.: deploy): gera o banco a
partir do Excel
        subprocess.run([sys.executable, str(ROOT / "src" / "etl.py")],
check=True)
    # conexão somente leitura: o app nunca consegue alterar os dados
    return sqlite3.connect(f"{DB.as_uri()}?mode=ro", uri=True,
check_same_thread=False)


@st.cache_data
def consulta(sql: str, params: tuple = ()) -> pd.DataFrame:
    return pd.read_sql_query(sql, conectar(), params=list(params))


def montar_where(filtros: dict, ini: str, fim: str) -> tuple[str, tuple]:
    """Colunas vêm de um dicionário fixo (nunca do usuário); valores vão como
parâmetros."""
    clausulas, params = ["data BETWEEN ? AND ?"], [ini, fim]
```

```
    for coluna, valores in filtros.items():
        clausulas.append(f"{coluna} IN ({','.join('?' * len(valores))})")
        params += valores
    return " AND ".join(clausulas), tuple(params)
```

O que este bloco ensina:

- `@st.cache_resource` guarda a conexão para não reabri-la a cada clique. `@st.cache_data` guarda o resultado das consultas.
- `mode=ro` abre o banco **somente leitura**. Mesmo que alguém digite `DROP TABLE` na aba SQL, o banco recusa.
- Se o `db/vendas.db` não existir (por exemplo, logo após o deploy), o app roda o ETL sozinho.
- `montar_where` monta o filtro dinamicamente. Os nomes das colunas vêm do nosso código, e os valores digitados pelo usuário vão sempre como parâmetros `?`.

Bloco 2: filtros da barra lateral

```
limites = consulta("SELECT MIN(data) AS ini, MAX(data) AS fim FROM
vw_vendas").iloc[0]
data_min, data_max = pd.to_datetime(limites["ini"]).date(),
pd.to_datetime(limites["fim"]).date()


def opcoes(coluna: str) -> list[str]:
    return consulta(f"SELECT DISTINCT {coluna} AS v FROM vw_vendas ORDER BY
1")["v"].tolist()


with st.sidebar:
    st.subheader("Filtros")
    vendedores = st.multiselect("Vendedor", opcoes("vendedor"),
default=opcoes("vendedor"))
    clientes = st.multiselect("Cliente", opcoes("cliente"),
default=opcoes("cliente"))
    produtos = st.multiselect("Produto", opcoes("produto"),
default=opcoes("produto"))
    regioes = st.multiselect("Região", opcoes("regiao"),
default=opcoes("regiao"))
    periodo = st.date_input("Período", value=(data_min, data_max),
                            min_value=data_min, max_value=data_max)

if not isinstance(periodo, tuple) or len(periodo) != 2:
    st.info("Selecione a data inicial e a final no filtro de período.")
    st.stop()
```

```
if not all([vendedores, clientes, produtos, regioes]):
    st.warning("Selecione ao menos uma opção em cada filtro.")
    st.stop()

where, params = montar_where(
    {"vendedor": vendedores, "cliente": clientes, "produto": produtos,
"regiao": regioes},
    periodo[0].isoformat(), periodo[1].isoformat(),
)

tab_dash, tab_sql, tab_modelo = st.tabs(["📊 Dashboard", "🧮 SQL ao vivo",
"🗄️ Modelo do banco"])
```

O que este bloco ensina:

- as opções dos filtros vêm do **próprio banco**, e não de listas escritas à mão;
- `st.stop()` interrompe a página de forma elegante quando o usuário ainda não terminou de escolher o período ou desmarcou tudo;
- `st.date_input` com dois valores permite escolher um intervalo, algo que o slider original não fazia.

Bloco 3: aba Dashboard

```
with tab_dash:
    st.header(":bar_chart: Dashboard de Vendas")

    kpi = consulta(
        f"""SELECT COALESCE(SUM(valor_pedido), 0) AS vendas,
                   COALESCE(SUM(margem_lucro), 0) AS lucro,
                   COUNT(*) AS pedidos
            FROM vw_vendas WHERE {where}""", params).iloc[0]

    if kpi["pedidos"] == 0:
        st.warning("Nenhum pedido para os filtros escolhidos.")
        st.stop()

    margem = 100 * kpi["lucro"] / kpi["vendas"]
    k1, k2, k3, k4 = st.columns(4)
    k1.metric("Vendas totais", f"R$ {kpi['vendas']:,.2f}")
    k2.metric("Total de lucro", f"R$ {kpi['lucro']:,.2f}")
    k3.metric("Margem média", f"{margem:.1f} %")
    k4.metric("Pedidos", f"{int(kpi['pedidos']):,}")

    def agrupar(expressao: str, apelido: str) -> pd.DataFrame:
        return consulta(
            f"""SELECT {expressao} AS {apelido},
```

```
                        SUM(quantidade) AS quantidade,
                        ROUND(SUM(valor_pedido), 2) AS valor_pedido
                    FROM vw_vendas WHERE {where}
                    GROUP BY 1 ORDER BY 1""", params)

    por_produto = agrupar("produto", "produto")
    por_cliente = agrupar("cliente", "cliente")
    por_vendedor = agrupar("vendedor", "vendedor")
    por_regiao = agrupar("regiao", "regiao")
    por_mes = agrupar("strftime('%Y-%m', data)", "mes")

    c1, c2, c3 = st.columns(3)
    with c1:
        st.altair_chart(
            alt.Chart(por_produto, title="Quantidade vendida por produto")
            .mark_bar(color="#9DD1F1")
            .encode(x="produto:N", y="quantidade:Q", tooltip=["produto",
"quantidade"]),
            use_container_width=True)
        st.altair_chart(
            alt.Chart(por_produto, title="Valor total por produto")
            .mark_bar(color="#9DD1F1")
            .encode(x="produto:N", y="valor_pedido:Q", tooltip=["produto",
"valor_pedido"]),
            use_container_width=True)
    with c2:
        st.altair_chart(
            alt.Chart(por_cliente, title="Valor vendido por cliente")
            .mark_bar(color="#666", cornerRadiusTopLeft=9,
cornerRadiusBottomRight=9)
            .encode(x=alt.X("cliente:N", sort="-y"), y="valor_pedido:Q",
                    tooltip=["cliente", "quantidade", "valor_pedido"]),
            use_container_width=True)
        st.altair_chart(
            alt.Chart(por_mes, title="Vendas mensais")
            .mark_line(color="#666", point=True)
            .encode(x=alt.X("mes:O", title="Mês"), y="valor_pedido:Q",
                    tooltip=["mes", "valor_pedido"]),
            use_container_width=True)
    with c3:
        st.altair_chart(
            alt.Chart(por_vendedor, title="Valor de venda por vendedor")
            .mark_arc(innerRadius=70, outerRadius=120)
            .encode(theta="valor_pedido:Q", color="vendedor:N",
                    tooltip=["vendedor", "valor_pedido"]),
            use_container_width=True)
```

```
        st.altair_chart(
            alt.Chart(por_regiao, title="Valor de venda por região")
            .mark_bar(color="#9DD1F1")
            .encode(y=alt.Y("regiao:N", sort="-x"), x="valor_pedido:Q",
                    tooltip=["regiao", "valor_pedido"]),
            use_container_width=True)
```

O que este bloco ensina:

- toda a conta (soma, contagem, agrupamento) é feita **pelo banco**, e não em Python. O Streamlit só desenha o resultado;
- `COALESCE(..., 0)` evita valor nulo quando os filtros não retornam nenhum pedido;
- a função `agrupar` repete o mesmo padrão de consulta para cinco gráficos diferentes. Sempre que você copiar e colar código três vezes, vale criar uma função;
- as letras `:N`, `:Q` e `:O` do Altair indicam o tipo do campo: nominal, quantitativo e ordinal.

Bloco 4: abas SQL ao vivo e Modelo do banco

```
with tab_sql:
    st.write("Consultas do arquivo `sql/queries.sql`. A conexão é **somente
leitura**: "
             "você pode editar e executar à vontade.")
    texto = (SQL_DIR / "queries.sql").read_text(encoding="utf-8")
    partes = re.split(r"^-- name:\s*(.+)$", texto, flags=re.M)[1:]
    consultas = dict(zip(partes[::2], (p.strip() for p in partes[1::2])))

    nome = st.selectbox("Consulta", list(consultas))
    sql = st.text_area("SQL", consultas[nome], height=240)
    if st.button("Executar", type="primary"):
        try:
            st.dataframe(pd.read_sql_query(sql.strip().rstrip(";"),
conectar()),
                         use_container_width=True, hide_index=True)
        except Exception as erro:
            st.error(f"Erro na consulta: {erro}")

with tab_modelo:
    st.subheader("Diagrama entidade-relacionamento")
    st.graphviz_chart("""
    digraph ER {
      rankdir=LR; node [shape=record, fontname="Helvetica"];
      pedido   [label="{pedido|numero_pedido
(PK)\\ldata\\lquantidade\\lpreco\\lmargem_lucro\\lvendedor_id
(FK)\\lcliente_id (FK)\\lproduto_id (FK)\\lregiao_id (FK)\\l}"];
```

```
      vendedor [label="{vendedor|id (PK)\\lnome\\l}"];
      cliente  [label="{cliente|id (PK)\\lnome\\l}"];
      produto  [label="{produto|id (PK)\\lnome\\l}"];
      regiao   [label="{regiao|id (PK)\\lnome\\l}"];
      vendedor -> pedido [label="1:N"]; cliente -> pedido [label="1:N"];
      produto  -> pedido [label="1:N"]; regiao  -> pedido [label="1:N"];
    }""")
    st.subheader("Script de criação (`sql/schema.sql`)")
    st.code((SQL_DIR / "schema.sql").read_text(encoding="utf-8"),
language="sql")
```

O que este bloco ensina:

- o app lê o `queries.sql` e separa as consultas pelo marcador `-- name:`, usando uma expressão regular;
- como a conexão é somente leitura, deixar o usuário editar o SQL é seguro: qualquer comando que altere dados falha com erro;
- o próprio app documenta o modelo, o que é ótimo para quem visita seu portfólio.

Passo 10: rodar e testar

Faça:

```
streamlit run app.py
```

O navegador abre em `http://localhost:8501`.

Confira, com todos os filtros no padrão:

Indicador	Valor esperado
Vendas totais	R$ 573,424.50 (o Streamlit usa o formato americano de número)
Total de lucro	R$ 88,349.00
Margem média	15.4 %
Pedidos	4,384

Teste os filtros: deixe somente **Ana** marcada em Vendedor. Os números devem mudar para 1.163 pedidos e R$ 158,306.55 em vendas. Depois desmarque todos os produtos e veja o aviso amarelo aparecer. Por fim, abra a aba **SQL ao vivo**, escolha uma consulta e clique em **Executar**.

Parte 8 — Versionando e publicando

Passo 11: subir para o Git

Faça: na raiz do projeto, depois de criar um repositório vazio no GitHub.

```
git init
git add .
git status
git commit -m "Projeto de banco de dados com SQLite e Streamlit"
git branch -M main
git remote add origin https://github.com/SEU_USUARIO/projeto_vendas.git
git push -u origin main
```

Confira: rode `git status` **antes** do commit e veja se `.venv/`, `pyvenv.cfg` e `db/` **não** aparecem na lista. Se aparecerem, o `.gitignore` está errado.

Passo 12: publicar no Streamlit Community Cloud

Explicação: o Streamlit Community Cloud hospeda o app de graça, mas só lê repositórios do **GitHub**.

Faça:

1. acesse share.streamlit.io e entre com sua conta do GitHub;
2. clique em **Create app** e escolha o repositório, a branch `main` e o arquivo `app.py`;
3. clique em **Deploy** e aguarde a instalação das dependências.

Confira: no primeiro acesso o app roda o ETL sozinho e gera o banco. Por isso o `data/system_extraction.xlsx` precisa estar no repositório.

Exercícios

Nível 1 — Consultas. Acrescente ao `queries.sql` uma consulta que mostre o faturamento por produto e por trimestre. Dica: use `strftime('%m', data)` e uma expressão `CASE`.

Nível 2 — Dashboard. Adicione um quinto cartão com o **ticket médio** (faturamento dividido pelo número de pedidos), respeitando os filtros.

Nível 3 — Modelagem. Crie uma tabela `meta_vendedor` (vendedor, mês e valor da meta). Carregue valores de exemplo no ETL e escreva uma consulta que compare realizado e meta.

Desafio — Desempenho. Rode `EXPLAIN QUERY PLAN` em uma consulta filtrando por `vendedor_id`, primeiro com o índice e depois sem ele. Explique a diferença.